

CITS5017 Deep Learning

Topic 1: Introduction to Artificial Neural Networks

Associate Professor Du Huynh
(Unit Coordinator and Lecturer)

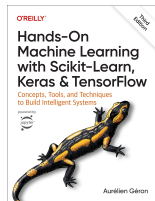
Semester 2, 2026



THE UNIVERSITY OF
WESTERN
AUSTRALIA

Chapters 10.

Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow



Introduction to Artificial Neural Networks

Here are the main topics we will cover:

- Perceptrons and *threshold logic unit* (TLU)
- Training of a Perceptron and *Hebb's rule*
- Perceptrons and the XOR classification problem
- Multi-Layer Perceptrons (MLP) and backpropagation
- Commonly used activation functions
- Training a deep neural network (DNN) using the Sequential API
- Training a deep neural network (DNN) using the Functional API
- Fine-tuning neural network hyperparameters

The Perceptron

- The *Perceptron* is one of the simplest ANN architectures, invented in 1957 by Frank Rosenblatt.
- It is based on a slightly different artificial neuron called a *threshold logic unit* (TLU), where the inputs and output are numbers (instead of binary on/off values) and each input connection is associated with a *weight*.
- The TLU computes a weighted sum of its inputs:
$$z = w_1x_1 + \dots + w_nx_n + b = \mathbf{w}^T\mathbf{x} + b.$$
It then applies a *step function* to that sum and outputs the result:
$$h_{\mathbf{w},b}(\mathbf{x}) = \text{step}(z).$$

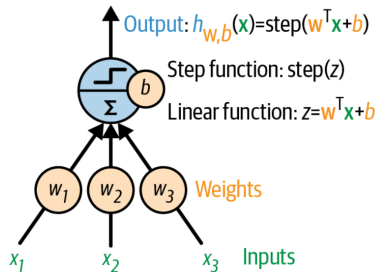


Figure 10-4. Threshold Logic Unit (TLU) or sometimes Linear Threshold Unit (LTU)

The Perception (cont.)

Step Functions

The most common step function used in Perceptrons is the *Heaviside step function*:

$$\text{heaviside}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

Sometimes the *sign function* is used instead:

$$\text{sign}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

The Perceptron (cont.)

A single TLU can be used for **simple linear binary classification**: It computes a linear combination of the inputs and if the result exceeds a threshold, it outputs the positive class or else outputs the negative class (just like a **Logistic Regression classifier** or a **linear SVM**).

A Perceptron is simply composed of one or more TLUs organised in a **single layer**, where

- every TLU is connected to every input. Such a layer is called a **fully connected layer**, or a **dense layer**;
- the inputs constitute the **input layer**; and
- since the layer of TLUs produces the final outputs, it is called the **output layer**.

Perceptron: An example

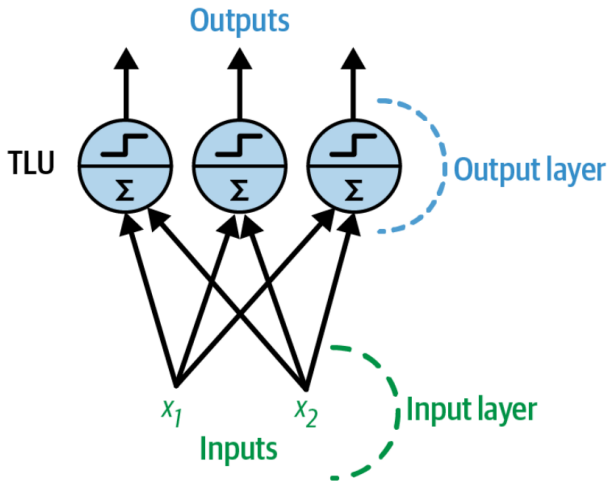


Figure 10-5. A Perceptron with two input and three output neurons.

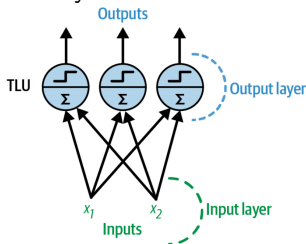
Perceptron: Computing the outputs

The magic of linear algebra makes it possible to efficiently compute the outputs of a fully connected layer:

$$h_{\mathbf{W}, \mathbf{b}}(\mathbf{X}) = \phi(\mathbf{X}\mathbf{W} + \mathbf{b}^\top)$$

where

- $\mathbf{X}$ represents the matrix of input features (one row per instance, one column per feature)
- $\mathbf{W}$ represents the unknown weight matrix which contains all the connection weights (one row per input feature and one column per neuron)
- $\mathbf{b}$ is the bias vector (one per neuron).
- ϕ is called the *activation function*



How is a Perceptron trained?

The Perceptron training algorithm proposed by Frank Rosenblatt was largely inspired by *Hebb's rule* or *Hebbian learning* (name after Donald Hebb) which can be summarized as “cells that fire together, wire together”.

More specifically, the Perceptron is fed one training instance at a time, and for each instance it makes its predictions. For every output neuron that produced a wrong prediction, it reinforces the connection weights from the inputs that would have contributed to the correct prediction.

How is a Perceptron trained? (cont.)

Perceptron learning rule (weight update):

$$w_{i,j}^{(\text{next step})} = w_{i,j} + \eta(y_j - \hat{y}_j)x_i$$

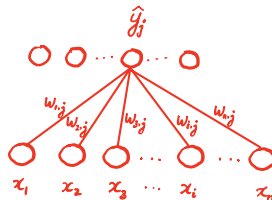
where

- $w_{i,j}$ is the connection weight between the i^{th} input neuron and the j^{th} output neuron.
- x_i is the i^{th} input value of the current training instance.
- $\hat{y}_j$ is the output of the j^{th} output neuron for the current training instance.
- y_j is the target output of the j^{th} output neuron for the current training instance.
- η is the learning rate.

How is a Perceptron trained? (cont.)

$$w_{i,j}^{(\text{next step})} = w_{i,j} + \eta(y_j - \hat{y}_j)x_i$$

$$w_{i,j}^{(\text{next step})} = w_{i,j} - \eta(\hat{y}_j - y_j)x_i$$



The loss that we try to minimise is $\frac{1}{2}(\hat{y}_j - y_j)^2$ where $\hat{y}_j = \sum_{i=1}^n x_i w_{i,j}$ (assuming that $W \in \mathbb{R}^{n \times m}$)

$$\underset{\text{(error)}}{L_j} = \frac{1}{2}(\hat{y}_j - y_j)^2 \Rightarrow \underset{\text{(error gradient w.r.t } w_{i,j})}{\frac{\partial L_j}{\partial w_{i,j}}} = (\hat{y}_j - y_j) \frac{\partial \hat{y}_j}{\partial w_{i,j}}$$

$$\text{Now } \frac{\partial \hat{y}_j}{\partial w_{i,j}} = \frac{\partial}{\partial w_{i,j}} \left(\sum_{i=1}^n x_i w_{i,j} \right) = \frac{\partial}{\partial w_{i,j}} (x_1 w_{1,j} + x_2 w_{2,j} + \dots + x_i w_{i,j} + \dots + x_n w_{n,j}) = x_i$$

$$\Rightarrow \frac{\partial L_j}{\partial w_{i,j}} = (\hat{y}_j - y_j) x_i$$

error gradient
for $w_{i,j}$

$$w_{i,j}^{(\text{next step})} = w_{i,j} - \eta \frac{\partial L_j}{\partial w_{i,j}}$$

negative error gradient
scaled by the learning rate.

Perceptron: Decision Boundary

The **decision boundary** of each output neuron is **linear**, so Perceptrons (just like Logistic Regression classifiers) are incapable of learning complex patterns.

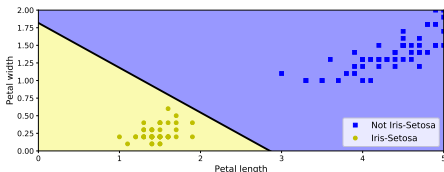
However, if the training instances are linearly separable, Rosenblatt demonstrated that this algorithm would converge to a solution.¹ This is called the *Perceptron convergence theorem*.

¹Note that this solution is generally not unique.

Implementing Perceptron in Scikit-Learn

Scikit-Learn provides a **Perceptron** class that implements a single TLU network:

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
iris = load_iris()
X = iris.data[:, (2, 3)]           # petal length, petal width
y = (iris.target == 0).astype(np.int) # Iris Setosa?
per_clf = Perceptron(random_state=42)
per_clf.fit(X, y)
y_pred = per_clf.predict([[2, 0.5], [3, 1]]) # predicts True and False
```



Decision boundary output by the code above.

Implementing Perceptron in Scikit-Learn (cont.)

The **Perceptron learning algorithm** strongly resembles **Stochastic Gradient Descent**. In fact, Scikit-Learn's **Perceptron** class is equivalent to using an **SGDClassifier** with the following hyperparameters:

```
loss="perceptron",  
learning_rate="constant",  
eta0=1 (learning rate), and  
penalty=None (no regularization).
```

Comparing Perceptron with Logistic Regression Classifiers

Note that contrary to Logistic Regression classifiers, **Perceptrons do not output a class probability**; rather, they just make predictions based on a hard threshold.

This is one of the good reasons to prefer Logistic Regression over Perceptrons.

Weaknesses of Perceptrons

In their 1969 monograph titled “*Perceptrons*”, Marvin Minsky and Seymour Papert highlighted some serious weaknesses of Perceptrons, in particular, their inability of solving some trivial problems (e.g., the *Exclusive OR (XOR)* classification problem).

Of course this is true of any other linear classification model as well (such as *Logistic Regression classifiers*), but researchers had expected much more from Perceptrons. As a result, many researchers dropped *connectionism* altogether.

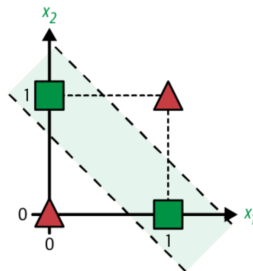


Figure 10-6a. XOR classification problem

Weaknesses of Perceptrons (cont.)

However, it turns out that some of the limitations of Perceptrons can be eliminated by stacking multiple Perceptrons. The resulting ANN is called a *Multi-Layer Perceptron* (MLP).

In particular, an MLP can solve the XOR problem, as you can verify by computing the output of the MLP for each combination of inputs:

- with inputs (0, 0) or (1, 1) the network outputs 0;
- with inputs (0, 1) or (1, 0) it outputs 1.

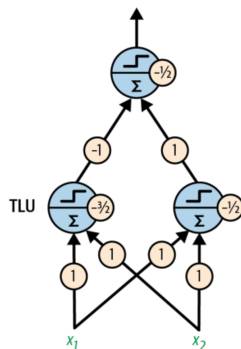


Figure 10-6b. An MLP that solves the XOR classification problem.

Multi-Layer Perceptron and Backpropagation

An **MLP** is composed of one (passthrough) *input layer*, one or more layers of TLUs, called *hidden layers*, and one final layer of TLUs called the *output layer*.

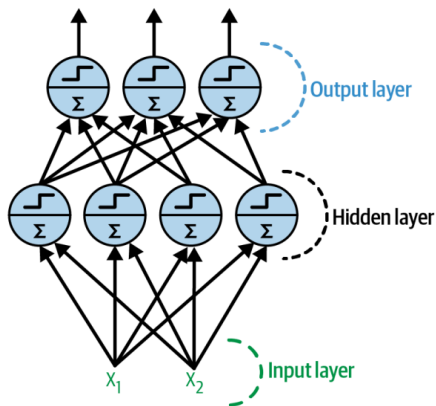


Figure 10-7. Multi-Layer Perceptron.

Multi-Layer Perceptron and Backpropagation (cont.)

We can describe the *backpropagation training algorithm*² as *Gradient Descent using reverse-mode autodiff*.

For each training instance, the algorithm

- feeds it to the network and computes the output of every neuron in each layer (this is the *forward pass*),
- measures the network's output error (i.e., the difference between the desired output and the actual output of the network),
- goes through each layer in reverse order to measure the error contribution from each connection (this is the *reverse pass*) until the input layer is reached, and
- finally slightly tweaks the connection weights to reduce the error (this is the *Gradient Descent step*).

²“Learning Internal Representations by Error Propagation,” D. Rumelhart, G. Hinton, R. Williams (1986). This algorithm was actually invented several times, starting with P. Werbos in 1974.

Multi-Layer Perceptron and Backpropagation (cont.)

Activation functions

In order for the backpropagation training algorithm to work, Rumelhart et al made a key change to the MLP's architecture: they replaced the step function with the **logistic function**, $\sigma(z) = 1/(1 + \exp(-z))$, also called the **sigmoid function**. This was essential because the step function has no gradient to work with, while the logistic function has a well-defined nonzero derivative everywhere.

Such a function which defines the output of a neuron (or node) given an input or set of inputs is known as the **activation function**.

(The derivative is:

$$\sigma'(z) = \frac{\exp(-z)}{(1+\exp(-z))^2} = \frac{1+\exp(-z)-1}{(1+\exp(-z))^2} = \frac{1}{1+\exp(-z)} - \frac{1}{(1+\exp(-z))^2} = \sigma(z) - \sigma^2(z) = \sigma(z)(1 - \sigma(z))$$

Multi-Layer Perceptron and Backpropagation (cont.)

Activation functions

Apart from the logistic function, two other popular **activation functions** are:

- *The hyperbolic tangent function:* $\tanh(z) = 2\sigma(2z) - 1$. This function has a similar S shape as the logistic function, is continuous and differentiable, but its values range from -1 to $+1$.
- *The rectified linear unit function:* $\text{ReLU}(z) = \max(0, z)$. This function is continuous but unfortunately not differentiable everywhere. However, in practice it works very well and has the advantage of being fast to compute. Most importantly, the fact that it does not have a maximum output value also helps reduce some issues during Gradient Descent.

Multi-Layer Perceptron and Backpropagation (cont.)

Activation functions

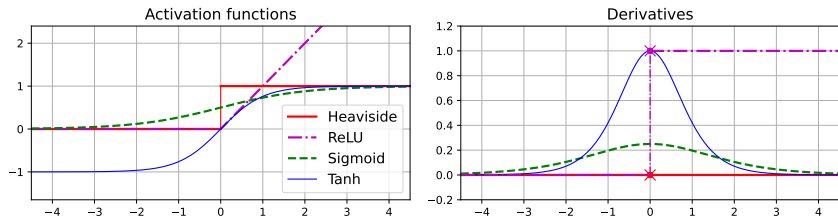


Figure 10-8. Activation functions and their derivatives

Multi-Layer Perceptron and Backpropagation (cont.)

Why do we need activation functions?

Consider two linear functions for your two layers $f(z) = 2z + 3$ and $g(z) = 5z - 1$. If you chain them together, you get $f(g(z)) = 2(5z - 1) + 3 = 10z + 1$, which is a linear function. If you don't have some nonlinearity between layers, then even a deep stack of layers is equivalent to a single layer, and you can't solve very complex problems with that.

Regression MLPs

MLPs can be used for regression tasks. For example, to predict the price of a house given many of its features, you need one output neuron. For multivariate regression, you need multiple neurons.

For regression, the output neurons usually do not need an activation function. Only when the output z needs to be

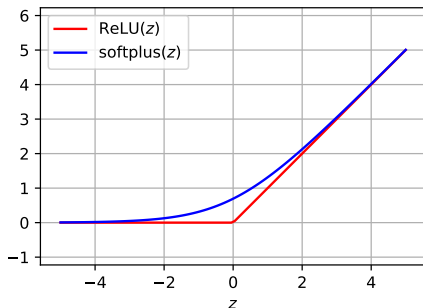
- always positive, then use the ReLU or $\text{softplus}(z) = \log(1 + \exp(z))$ activation function;
- in a certain range of values, then use logistic or hyperbolic tangent and then scale the value to the required range.

Regression MLPs (cont.)

The **softplus** function is like the **ReLU** function except that it is smooth at the origin:

$$\text{ReLU}(z) = \max(0, z)$$

$$\text{softplus}(z) = \log(1 + \exp(z))$$



Regression MLPs (cont.)

An example

```
from sklearn.datasets import fetch_california_housing
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

housing = fetch_california_housing()
X_train_full, X_test, y_train_full, y_test = train_test_split(
    housing.data, housing.target, random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train_full, y_train_full, random_state=42)

# 3 hidden layers with 50 neurons per layer
mlp_reg = MLPRegressor(hidden_layer_sizes=[50, 50, 50], random_state=42)
pipeline = make_pipeline(StandardScaler(), mlp_reg)
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_valid)
rmse = mean_squared_error(y_valid, y_pred, squared=False) # around 0.505
```

The median house prices (the *y* values) have been normalised to the range [0, 5], so the RMSE 0.505 is equivalent to around 10% error.

Regression MLPs (cont.)

The loss function to use during training is typically the **mean squared error** or **mean absolute error** (the latter is more robust against outliers). Alternatively, the **Huber loss**, which is a combination of MSE and MAE, can also be used.

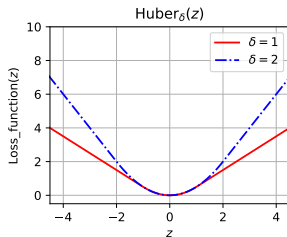
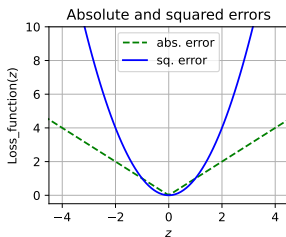
Typical regression MLP architecture:

Hyperparameter	Typical value
# hidden layers	Depends on the problem, but typically 1 to 5
# neurons per hidden layer	Depends on the problem, but typically 10 to 100
# output neurons	1 per prediction dimension
Hidden activation	ReLU
Output activation	None, or ReLU/softplus (if positive outputs) or sigmoid/tanh (if bounded outputs)
Loss function	MSE, or Huber if outliers

Regression MLPs (cont.)

The *Huber* loss behaves like the *absolute error* loss when the input z is large and behaves like the *squared error* loss when the input z is small.

$$\text{Huber}_\delta(z) = \begin{cases} \frac{1}{2}z^2 & \text{if } |z| \leq \delta \\ \delta (|z| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$



Classification MLPs

MLPs can also be used for classification tasks.

- For **binary classification** problems, one single output neuron with a *logistic activation function* would be sufficient.
- For **multilabel classification** problems, you can have multiple output neurons, each with a *logistic activation function*.
- For **multiclass classification** problems (e.g., MNIST dataset), you can have multiple output neurons, with the *softmax activation function*. This will ensure that all the output values are probabilities and they sum to 1.

For the loss function, since we are predicting probability distributions, the *cross-entropy* loss (also called the *log loss*) is generally a good choice:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$$

where K denotes the total number of classes (e.g., $K = 10$ for MNIST).

Classification MLPs (cont.)

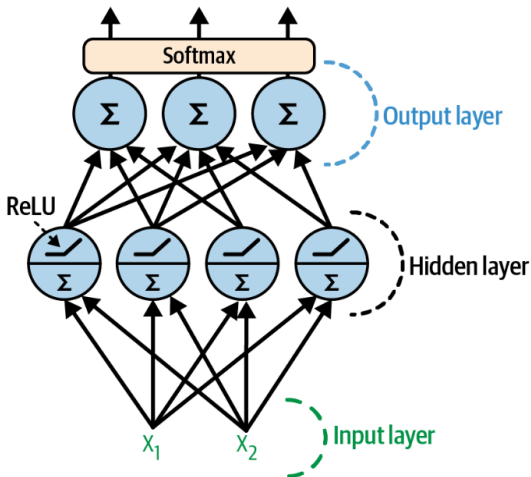


Figure 10-9. A modern MLP (including ReLU and softmax) for classification

Classification MLPs (cont.)

Typical classification MLP architecture:

Hyperparameter	Binary classification	Multilabel binary classification	Multiclass classification
# hidden layers	Typically 1 to 5 layers, depending on the task		
# output neurons	1	1 per binary label	1 per class
Output layer activation	Sigmoid	Sigmoid	Softmax
Loss function	X-entropy	X-entropy	X-entropy

Implementing MLPs with Keras

Keras (<https://keras.io/>) is a high-level Deep Learning API that allows you to easily build, train, evaluate, and execute all sorts of neural networks.

It was developed by François Chollet and was released as an open source project in March 2015. It quickly gained popularity and, at present, is available in three popular open source Deep Learning libraries: *TensorFlow*, *Microsoft Cognitive Toolkit* (CNTK), and *Theano*. We will refer to this reference implementation as *multibackend Keras*. However, since version 2.4, Keras is TensorFlow-only.

Implementing MLPs with Keras (cont.)

After installation, check that you have the correct version of TensorFlow for the unit:

```
import tensorflow as tf
from tensorflow import keras

tf.__version__
# output: '2.17.0'

keras.__version__
# output: '3.10.0'
```

Example: Building an Image Classifier Using the Sequential API

A linear model can reach about 92% on MNIST, but only about 83% on *Fashion MNIST*, which also has 10 classes with input being 28×28 images:



Figure 10-11. Samples from Fashion MNIST.

The 10 classes are: "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat", "Sandal", "Shirt", "Sneaker", "Bag", and "Ankle boot".

Example: Building an Image Classifier Using the Sequential API (cont.)

Loading the data and creating the model

```
import tensorflow as tf

fashion_mnist = tf.keras.datasets.fashion_mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist #60,000 and 10,000
# split training set to form a validation set (last 5000 instances)
X_train, y_train = X_train_full[:-5000], y_train_full[:-5000]
X_valid, y_valid = X_train_full[-5000:], y_train_full[-5000:]
# scale pixel intensities to the 0..1 range
X_train, X_valid, X_test = X_train / 255., X_valid / 255., X_test / 255.

# creating the model
model = tf.keras.Sequential()
model.add(tf.keras.layers.InputLayer(shape=[28, 28]))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(300, activation="relu"))
model.add(tf.keras.layers.Dense(100, activation="relu"))
model.add(tf.keras.layers.Dense(10, activation="softmax"))
```

The last few lines above can be replaced by

```
model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[28, 28]),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(300, activation="relu"),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

Example: Building an Image Classifier Using the Sequential API (cont.)

Inspecting the model

```
>>> model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_3 (Dense)	(None, 300)	235,500
dense_4 (Dense)	(None, 100)	30,100
dense_5 (Dense)	(None, 10)	1,010

Total params: 266,610 (1.02 MB)

Trainable params: 266,610 (1.02 MB)

Non-trainable params: 0 (0.00 B)

```
>>> hidden1 = model.layers[1]
>>> hidden1.name
'dense_3'
>>> weights, biases = hidden1.get_weights()
.....
>>> weights.shape
(784, 300)
>>> biases.shape
(300,)
```

Example: Building an Image Classifier Using the Sequential API (cont.)

Compiling the model

```
model.compile(loss="sparse_categorical_crossentropy", optimizer="sgd", metrics=["accuracy"])
```

Training and evaluating the model

```
>>> history = model.fit(X_train, y_train, epochs=30, validation_data=(X_valid, y_valid))
```

```
1719/1719 ————— 5s 2ms/step - loss: 1.0023 - sparse_categorical_accuracy: 0.6816 - val_loss: 0.5075 - val_sparse_cate
gorical_accuracy: 0.8246
Epoch 2/30
1719/1719 ————— 3s 2ms/step - loss: 0.5057 - sparse_categorical_accuracy: 0.8262 - val_loss: 0.4561 - val_sparse_cate
gorical_accuracy: 0.8370
Epoch 3/30
1719/1719 ————— 3s 2ms/step - loss: 0.4536 - sparse_categorical_accuracy: 0.8418 - val_loss: 0.4306 - val_sparse_cate
gorical_accuracy: 0.8446
Epoch 4/30
1719/1719 ————— 3s 2ms/step - loss: 0.4236 - sparse_categorical_accuracy: 0.8514 - val_loss: 0.4160 - val_sparse_cate
gorical_accuracy: 0.8514
. . . . .
Epoch 27/30
1719/1719 ————— 3s 2ms/step - loss: 0.2352 - sparse_categorical_accuracy: 0.9161 - val_loss: 0.3454 - val_sparse_cate
gorical_accuracy: 0.8762
Epoch 28/30
1719/1719 ————— 3s 2ms/step - loss: 0.2311 - sparse_categorical_accuracy: 0.9180 - val_loss: 0.3451 - val_sparse_cate
gorical_accuracy: 0.8760
Epoch 29/30
1719/1719 ————— 3s 2ms/step - loss: 0.2270 - sparse_categorical_accuracy: 0.9195 - val_loss: 0.3456 - val_sparse_cate
gorical_accuracy: 0.8768
Epoch 30/30
1719/1719 ————— 3s 2ms/step - loss: 0.2231 - sparse_categorical_accuracy: 0.9217 - val_loss: 0.3441 - val_sparse_cate
gorical_accuracy: 0.8776
```

Useful options to include when calling the `fit` method:
`class_weight`, `sample_weight`, `batch_size` (default=32)

Example: Building an Image Classifier Using the Sequential API (cont.)

Inspecting the learning curves

```
import matplotlib.pyplot as plt
import pandas as pd

pd.DataFrame(history.history).plot(
    figsize=(8, 5), xlim=[0, 29], ylim=[0, 1], grid=True, xlabel="Epoch",
    style=["r--", "r--.", "b-", "b-*"])
plt.legend(loc="lower left") # extra code
plt.show()
```

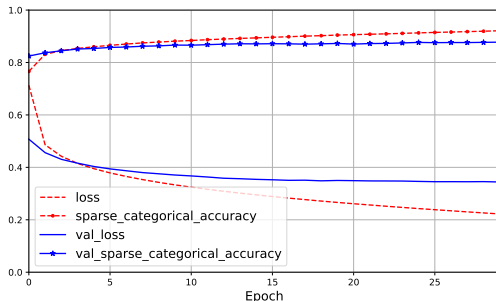


Figure 10-11. Learning curves.

Example: Building an Image Classifier Using the Sequential API (cont.)

Applying the trained model on the test set

```
>>> model.evaluate(X_test, y_test)
313/313 [=====] - 0s 903us/step - accuracy: 0.8738 - loss: 0.3715
[0.37079566717147827, 0.8733000159263611]
```

Example: Building an Image Classifier Using the Sequential API (cont.)

Using the model to make predictions

```
>>> X_new = X_test[:3]
>>> y_proba = model.predict(X_new)
>>> y_proba.round(2)
1/1 ===== 0s 44ms/step
array([[0. , 0. , 0. , 0. , 0. , 0.18, 0. , 0.01, 0. , 0.8 ],
       [0. , 0. , 1. , 0. , 0. , 0. , 0. , 0. , 0. , 0. ],
       [0. , 1. , 0. , 0. , 0. , 0. , 0. , 0. , 0. , 0. ]],
      dtype=float32)

>>> import numpy as np
>>> y_pred = y_proba.argmax(axis=-1)
>>> y_pred
array([9, 2, 1])
>>> np.array(class_names)[y_pred]
array(['Ankle boot', 'Pullover', 'Trouser'], dtype='<U11')

# compare with the ground truth
>>> y_new = y_test[:3]
>>> y_new
array([9, 2, 1], dtype=uint8)
```

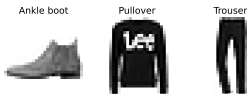


Figure 10-12. Correctly classified Fashion MNIST images.

Example: Building a Regression MLP Using the Sequential API

Example: the California housing problem

```
housing = fetch_california_housing()
X_train_full, X_test, y_train_full, y_test = train_test_split(
    housing.data, housing.target, random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train_full, y_train_full, random_state=42)

tf.random.set_seed(42)
norm_layer = tf.keras.layers.Normalization()
model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=X_train.shape[1:]),
    norm_layer,
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(1)
])
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer, metrics=["RootMeanSquaredError"])
norm_layer.adapt(X_train)
history = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_valid, y_valid))
mse_test, rmse_test = model.evaluate(X_test, y_test)  # rmse_test was around 0.539
X_new = X_test[:3]
y_pred = model.predict(X_new)
```

Example: Building a Regression MLP Using the Sequential API (cont.)

A close inspection on the output shows that this simple MLP does not produce very good predictions. Sometimes it is useful to build neural networks with more complex topologies, such as using the *Functional API*.

Building Complex Models Using the Functional API

One example of a non-sequential neural network is a *Wide & Deep* neural network, an architecture introduced in 2016[†]. This architecture makes it possible for the neural network to learn both deep patterns (using the deep path) and simple rules (through the short path).

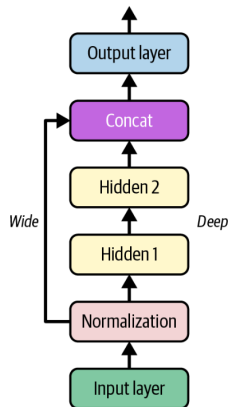


Figure 10-13. Wide & Deep neural network

[†]Heng-Tze Cheng et al., "Wide & Deep Learning for Recommender Systems", Proc. of the First Workshop on Deep Learning for Recommender Systems (2016): 7-10

Building Complex Models Using the Functional API (cont.)

The California Housing example

```
normalization_layer = tf.keras.layers.Normalization()
hidden_layer1 = tf.keras.layers.Dense(30, activation="relu")
hidden_layer2 = tf.keras.layers.Dense(30, activation="relu")
concat_layer = tf.keras.layers.Concatenate()
output_layer = tf.keras.layers.Dense(1)

input_ = tf.keras.layers.Input(shape=X_train.shape[1:])
normalized = normalization_layer(input_)
hidden1 = hidden_layer1(normalized)
hidden2 = hidden_layer2(hidden1)
concat = concat_layer([normalized, hidden2])
output = output_layer(concat)

model = tf.keras.Model(inputs=[input_], outputs=[output])
```

Building Complex Models Using the Functional API (cont.)

The California Housing example

```
>>> model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 8)	0	-
normalization (Normalization)	(None, 8)	17	input_layer[0][0]
dense (Dense)	(None, 30)	270	normalization[0]...
dense_1 (Dense)	(None, 30)	930	dense[0][0]
concatenate (Concatenate)	(None, 38)	0	normalization[0]... dense_1[0][0]
dense_2 (Dense)	(None, 1)	39	concatenate[0][0]

Total params: 1,256 (4.91 KB)

Trainable params: 1,239 (4.84 KB)

Non-trainable params: 17 (72.00 B)

Building Complex Models Using the Functional API (cont.)

Multiple inputs. What if we want a different set of features through the *wide path* and the *deep path*?

```
# Using the California House dataset as an example
input_wide = tf.keras.layers.Input(shape=[5]) # features 0 to 4
input_deep = tf.keras.layers.Input(shape=[6]) # features 2 to 7
norm_layer_wide = tf.keras.layers.Normalization()
norm_layer_deep = tf.keras.layers.Normalization()
norm_wide = norm_layer_wide(input_wide)
norm_deep = norm_layer_deep(input_deep)
hidden1 = tf.keras.layers.Dense(30, activation="relu")(norm_deep)
hidden2 = tf.keras.layers.Dense(30, activation="relu")(hidden1)
concat = tf.keras.layers.concatenate([norm_wide, hidden2])
output = tf.keras.layers.Dense(1)(concat)
model = tf.keras.Model(inputs=[input_wide, input_deep], outputs=[output])

optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer,
              metrics=["RootMeanSquaredError"])
X_train_wide, X_train_deep = X_train[:, :5], X_train[:, 2:]
X_valid_wide, X_valid_deep = X_valid[:, :5], X_valid[:, 2:]
X_test_wide, X_test_deep = X_test[:, :5], X_test[:, 2:]
X_new_wide, X_new_deep = X_test_wide[:3], X_test_deep[:3]

norm_layer_wide.adapt(X_train_wide)
norm_layer_deep.adapt(X_train_deep)
history = model.fit((X_train_wide, X_train_deep), y_train, epochs=20,
                  validation_data=((X_valid_wide, X_valid_deep), y_valid))
mse_test = model.evaluate((X_test_wide, X_test_deep), y_test)
y_pred = model.predict((X_new_wide, X_new_deep))
```

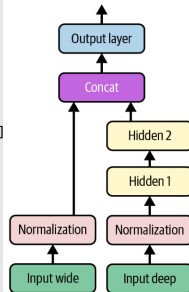


Figure 10-14. W&DNN with multiple inputs

Building Complex Models Using the Functional API (cont.)

Multiple inputs and multiple outputs.

We can also have multiple outputs from the model. Adding an extra output is easy: we just connect it to the appropriate layer and add it to the model's list of outputs. For example:

```
[...] # Same as above, up to the main output layer
output = tf.keras.layers.Dense(1)(concat)
aux_output = tf.keras.layers.Dense(1)(hidden2)
model = tf.keras.Model(inputs=[input_wide, input_deep],
                       outputs=[output, aux_output])

[...]
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss=("mse", "mse"), loss_weights=(0.9, 0.1),
              optimizer=optimizer,
              metrics=["RootMeanSquaredError", "RootMeanSquaredError"])
```

In this example, the *deep* branch has a more complex architecture. So, it is more likely to overfit the data than the *wide* branch. Its loss is therefore weighted by 0.1. So the auxiliary output is used for regularization.

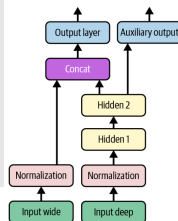


Figure 10-15. W&DNN with multiple inputs and multiple outputs

Using the Subclassing API to Build Dynamic Models

If you prefer a more imperative programming style, you can subclass the `Model` class. e.g., (compare with the code on Slide 46)

```
class WideAndDeepModel(tf.keras.Model):
    def __init__(self, units=30, activation="relu", **kwargs):
        super().__init__(**kwargs) # needed to support naming the model
        self.norm_layer_wide = tf.keras.layers.Normalization()
        self.norm_layer_deep = tf.keras.layers.Normalization()
        self.hidden1 = tf.keras.layers.Dense(units, activation=activation)
        self.hidden2 = tf.keras.layers.Dense(units, activation=activation)
        self.output = tf.keras.layers.Dense(1)

    def call(self, inputs):
        input_wide, input_deep = inputs
        norm_wide = self.norm_layer_wide(input_wide)
        norm_deep = self.norm_layer_deep(input_deep)
        hidden1 = self.hidden1(norm_deep)
        hidden2 = self.hidden2(hidden1)
        concat = tf.keras.layers.concatenate([norm_wide, hidden2])
        return self.output(concat)

model = WideAndDeepModel(30, activation="relu", name="my_cool_model")

optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer,
              metrics=["RootMeanSquaredError"])
model.norm_layer_wide.adapt(X_train_wide)
model.norm_layer_deep.adapt(X_train_deep)
history = model.fit(...)
```

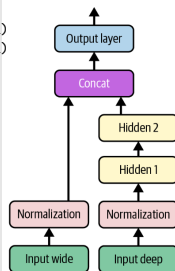


Figure 10-14. W&DNN with multiple inputs

Using the Subclassing API to Build Dynamic Models (cont.)

- You can include anything you want in the `call()` method: *for* loops, *if* statements, low-level TensorFlow operations, etc. This makes it a great API when experimenting with new ideas, especially for researchers.
- However, your model's architecture is hidden within the `call()` method and Keras cannot inspect it. Your model cannot be cloned using `tf.keras.models.clone_model()`. When you call the `summary()` method, you cannot see how the layers are connected in your model.

Saving and Restoring a Model

For both the Sequential API and the Functional API, we start by declaring the layers we want to use and how they should be connected. The actual feeding of data for training is done after the neural network has been constructed. This has the advantages that the model can be saved, cloned, and shared.

Saving a trained Keras model is very simple:

```
model = tf.keras.Sequential([...])  
model.compile([...])  
model.fit([...])  
model.save("my_keras_model.keras")
```

Restoring a model from disk is equally simple:

```
loaded_model = tf.keras.models.load_model("my_keras_model.keras")  
y_pred = loaded_model.predict(X_test)
```

Saving and Restoring a Model (cont.)

Using Callbacks

If your model takes a long time to train, it is useful to save *checkpoints* at regular intervals during training. To do so, we use *callbacks* to tell the `fit()` method to save checkpoints.

```
checkpoint_cb = tf.keras.callbacks.ModelCheckpoint(
    "my_checkpoints.weights.h5", save_best_only=True)
history = model.fit(..., validation_data=..., callbacks=[checkpoint_cb])
```

By default, a model checkpoint is saved at the end of each epoch. Many other callbacks are available in the `tf.keras.callbacks` package.

Saving and Restoring a Model (cont.)

Using Callbacks

Another way to implement early stopping is to simply use the `EarlyStopping` callback to interrupt training when no progress is observed on the validation set for a number of epochs (defined by the `patience` argument):

```
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=10,  
                                                    restore_best_weights=True)  
history = model.fit(X_train, y_train, epochs=100,  
                  validation_data=(X_valid, y_valid),  
                  callbacks=[checkpoint_cb, early_stopping_cb])
```

Using TensorBoard for Visualization

TensorBoard is a great interactive visualization tool for viewing the learning curves during training, compare learning curves between multiple runs, etc. It comes with the installation of TensorFlow.

To use TensorBoard, your program needs to output the data that you want to visualize to special binary log files called *event files*. We need to specify a root log directory for storing our TensorBoard logs. We can also include extra information in the log directory name, such as hyperparameter values that you are testing for inspection in TensorBoard:

```
from pathlib import Path
from time import strftime

def get_run_logdir(root_logdir="my_logs"):
    return Path(root_logdir) / strftime("run_%Y_%m_%d_%H_%M_%S")

run_logdir = get_run_logdir()
```

Using TensorBoard for Visualization (cont.)

Keras provides a convenient `TensorBoard()` callback that will take care of creating the log directory for you, and it will create event files and write summaries to them during training:

```
[...] # Build and compile your model
tensorboard_cb = tf.keras.callbacks.TensorBoard(run_logdir,
                                                profile_batch=(100, 200))
history = model.fit(..., callbacks=[tensorboard_cb])
```

If you run your code again, with a new log subdirectory, you will get a new log subdirectory with the old subdirectory.

Using TensorBoard for Visualization (cont.)

To start TensorBoard inside Jupyter, load the Jupyter extension for TensorBoard:

```
%load_ext tensorboard  
%tensorboard --logdir=./my_logs
```

Alternatively, you can simply type in a terminal window (outside Jupyter):

```
tensorboard --logdir=./my_logs
```

Once the server has been started, open a web browser and go to <http://localhost:6006>.

Using TensorBoard for Visualization (cont.)

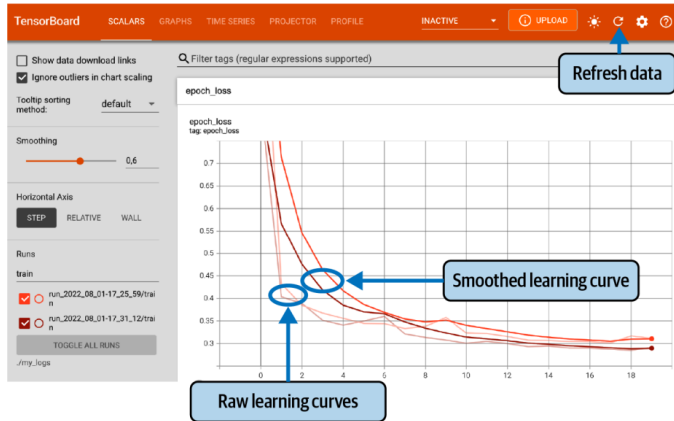


Figure 10-16. Visualizing learning curves with TensorBoard.

Fine-Tuning Neural Network Hyperparameters

The flexibility of neural networks is also one of their main drawbacks: there are many hyperparameters to tweak. We can use `GridSearchCV` or `RandomizedSearchCV` to explore the hyperparameter space.

you can use the *Keras Tuner* library, which is a hyperparameter tuning library for Keras models. It offers several tuning strategies.

Despite all this exciting progress and all these tools and services, it still helps to have an idea of what values are reasonable for each hyperparameter so that you can build a quick prototype and restrict the search space.

The remaining slides provide guidelines for choosing some of the main hyperparameters.

Fine-Tuning Neural Network Hyperparameters (cont.)

Number of Hidden Layers

- For many problems you can start with just one or two hidden layers and the neural network will work just fine, e.g., 97% (or above 98%) accuracy can be reached on the MNIST dataset using just one (or two) hidden layer(s) with a few hundred neurons.
- For more complex problems, you can increase the number of hidden layers until you just start overfitting the training set – deep networks have a much higher *parameter efficiency* than shallow ones. Typically, for complex image recognition or speech recognition tasks, dozens (maybe hundreds) of hidden layers (but not fully connected) are required.
- Can also reduce the number hidden layers by training a network that has been pre-trained on a similar problem. This is known as *transfer learning*.

Fine-Tuning Neural Network Hyperparameters (cont.)

Number of Neurons per Hidden Layer

- Obviously the number of neurons in the input and output layers is determined by the type of input and output your task requires.
- As for the hidden layers, a common practice is to size them to form a **pyramid**, with fewer and fewer neurons at each layer – the rationale being that many low-level features can coalesce into far fewer high-level features. However, a common practice today is to have the same number of neurons in each hidden layer. This also helps to reduce the number of parameters to tune.
- Just like the number of layers, you can try increasing the number of neurons gradually until the network starts overfitting, then use early stopping and other regularization techniques to prevent it from overfitting.

Fine-Tuning Neural Network Hyperparameters (cont.)

Learning Rate

Learning rate is arguably the most important hyperparameter. In general, the optimal learning rate is about half of the maximum learning rate (i.e., the learning rate above which the training algorithm diverges). One way to find a good learning rate is to start with a very low learning rate and gradually increase it to a very large value, e.g., increasing it from 10^{-5} to 10 in 500 iterations. If you plot the loss versus the learning rate, you should see the loss going down; at the point where the loss starts to shoot up, the optimal learning rate is about 10 times lower than the learning rate there.

Optimizer

Choosing a better optimizer than plain old mini-batch gradient descent (and tuning its hyperparameters) is also quite important. We will examine several advanced optimizers in Chapter 11.

Fine-Tuning Neural Network Hyperparameters (cont.)

Batch size

The common recommendation is to use larger batch size that can fit in the GPU RAM; however, in practice, large batch sizes often lead to training instabilities, especially at the beginning of training, and the resulting model may not generalize as well as a model trained with a small batch size. Currently, small batch sizes (2 to 32) are preferred by some researchers while large batch sizes are recommended by others. *Learning rate warmup* has been suggested to use with large batch sizes.

Fine-Tuning Neural Network Hyperparameters (cont.)

Activation Functions

In most cases you can use the **ReLU** activation function in the hidden layers (or one of its variants). It is a bit faster to compute than other activation functions, and Gradient Descent does not get stuck as much on plateaus (as opposed to the logistic function or the hyperbolic tangent function, which saturate at 1). For the output layer, it really depends on your task.

Number of iterations

In most cases, the number of training iterations does not actually need to be tweaked: just use early stopping instead.

Summary

- Understand the architecture of Perceptrons
- Know how to train a Perceptron and understand its weakness
- Understand multi-layer Perceptrons and backpropagation
- Understand what activation functions are and know a few commonly used activation functions
- Know how to train a DNN using Keras in TensorFlow
- Understanding the MLP sequential API, function API, and subclassing.
- Understand how to use TensorBoard
- Understand some of the important hyperparameters in a DNN; in particular, the number of hidden layers, the number of neurons per layer, and the activation functions.

For the next topic...

Read up to Chapter 11 on **Training Deep Neural Networks**

And that's all for this lecture.

Have a good week.